MongoDB und NoSQL: Intro

Dokumentenmodell, CRUD, Aggregation

Was ist NoSQL?

- Sammelbegriff für Datenbanksysteme, die Daten nicht primär relational organisieren.
- Typische Ziele sind flexible Modellierung, horizontale Skalierung und verteilte Verarbeitung.
- Viele Systeme sind schemaflexibler als klassische relationale Datenbanken.
- NoSQL ersetzt SQL nicht pauschal, sondern ergänzt es für andere Problemklassen.

NoSQL-Typen

- **Dokumentenorientiert**: MongoDB, Couchbase
- **Key-Value**: Redis, DynamoDB
- **Wide-Column**: Cassandra, HBase
- **Graph**: Neo4j, JanusGraph

Praxisregel:

- Zuerst Datenzugriff und Änderungsverhalten verstehen, erst danach den Datenbanktyp auswählen.

Warum MongoDB?

- BSON-Dokumente bilden Anwendungsobjekte sehr direkt ab.
- Verschachtelte Objekte und Arrays lassen sich natürlich modellieren.
- Das Schema kann mit dem Produkt wachsen, ohne bei jeder Änderung Migrationen zu erzwingen.
- Shell, Treiber und Ökosystem machen MongoDB für Teams pragmatisch nutzbar.

Datenmodell in MongoDB

Kernbegriffe:

- **Database**: fachlicher Container
- **Collection**: Gruppe ähnlicher Dokumente
- **Document**: zusammengehörige Daten in einer Einheit
- **Field**: einzelnes Attribut im Dokument

Dokumente können:

- verschachtelte Objekte enthalten,
- Arrays enthalten,
- je nach Fall leicht unterschiedliche Felder besitzen.

Beispiel-Dokument

```json
{
  "_id": "65fd1e...",
  "matrikelnummer": "S-2026-001",
  "name": {
    "vorname": "Anna",
    "nachname": "Schmidt"
  },
  "gruppe": "INF-21A",
  "tags": ["python", "tutorin"],
  "kurs_ids": ["DB-101", "AI-210"],
  "credits": 84
}
```

Embedded vs Referencing

Embedded (einbetten)

- sinnvoll, wenn Daten gemeinsam gelesen und gemeinsam gespeichert werden
- reduziert zusätzliche Abfragen
- vermeidet technische Joins in vielen Alltagsszenarien
- wird problematisch, wenn Teilobjekte stark wachsen oder unabhängig weiterverwendet werden

Referencing (referenzieren)

- sinnvoll, wenn Entitäten mehrfach genutzt werden
- reduziert Redundanz und vereinfacht zentrale Änderungen
- benötigt oft zusätzliche Abfragen oder `$lookup`
- passt gut zu Kursen, Produkten, Rollen oder Stammdaten

Referencing in der Praxis

```
// students
{
  matrikelnummer: "S-2026-001",
  name: { vorname: "Anna", nachname: "Schmidt" },
  kurs_ids: ["DB-101", "AI-210"]
}

// kurse
{
  kurs_id: "DB-101",
  titel: "Datenbanken",
  ects: 5
}
```

> Viele Studierende belegen dieselben Kurse, deshalb wäre Einbetten hier meist unnötig redundant.

Arbeitskontext in der Shell

```
mongosh
use workshop
db.createCollection("students")
db.createCollection("kurse")
show collections
```

- Alle folgenden Befehle laufen in der MongoDB-Shell `mongosh`.
- `use workshop` wechselt in die Datenbank und legt sie bei Bedarf an.
- Collections entstehen explizit oder implizit beim ersten Schreiben.

CRUD in MongoDB

```
// C: Create
db.students.insertOne({
  matrikelnummer: "S-2026-002",
  name: { vorname: "Lukas", nachname: "Weber" },
  gruppe: "INF-21A",
  tags: ["java"],
  kurs_ids: ["DB-101"],
  credits: 72
});

// R: Read
db.students.find(
  { gruppe: "INF-21A" },
  { _id: 0, matrikelnummer: 1, "name.vorname": 1, "name.nachname": 1, credits: 1 }
);

// U: Update
db.students.updateOne(
  { matrikelnummer: "S-2026-002" },
  {
    $set: { credits: 78 },
    $addToSet: { tags: "mongodb" }
  }
);

// D: Delete
db.students.deleteOne({ matrikelnummer: "S-2026-002" });
```

Abfragen und Filter

```
// Vergleichsoperatoren
db.students.find({ credits: { $gte: 60, $lt: 90 } });

// Kombination von Bedingungen
db.students.find({
  gruppe: "INF-21A",
  tags: "python"
});

// Sortierung, Projektion und Limit
db.students.find(
  { gruppe: "INF-21A" },
  { _id: 0, matrikelnummer: 1, "name.vorname": 1, credits: 1 }
).sort({ credits: -1 }).limit(5);
```

Aggregation Pipeline

```
db.students.aggregate([
  {
    $match: {
      gruppe: { $in: ["INF-21A", "INF-21B"] }
    }
  },
  {
    $group: {
      _id: "$gruppe",
      durchschnittliche_credits: { $avg: "$credits" },
      anzahl_studierende: { $sum: 1 }
    }
  },
  {
    $sort: {
      durchschnittliche_credits: -1
    }
  }
]);
```

Join-ähnliche Auswertung mit `$lookup`

```
db.students.aggregate([
  { $match: { matrikelnummer: "S-2026-001" } },
  {
    $lookup: {
      from: "kurse",
      localField: "kurs_ids",
      foreignField: "kurs_id",
      as: "kurse"
    }
  },
  {
    $project: {
      _id: 0,
      matrikelnummer: 1,
      "name.vorname": 1,
      "name.nachname": 1,
      "kurse.titel": 1,
      "kurse.ects": 1
    }
  }
]);
```

Indexe in MongoDB

```
// Eindeutige Identifikation
db.students.createIndex({ matrikelnummer: 1 }, { unique: true });

// Häufige Liste: Gruppe plus Sortierung nach Credits
db.students.createIndex({ gruppe: 1, credits: -1 });

// Referenzsammlung
db.kurse.createIndex({ kurs_id: 1 }, { unique: true });
```

- Indexe beschleunigen Lesezugriffe, kosten aber Speicher und verlangsamen Schreiboperationen.
- Gute Indexe leiten sich aus echten Query-Mustern ab, nicht aus Bauchgefühl.

Schema-Validierung (optional, aber sinnvoll)

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["matrikelnummer", "name", "gruppe"],
      properties: {
        matrikelnummer: { bsonType: "string" },
        name: {
          bsonType: "object",
          required: ["vorname", "nachname"],
          properties: {
            vorname: { bsonType: "string" },
            nachname: { bsonType: "string" }
          }
        },
        gruppe: { bsonType: "string" },
        tags: {
          bsonType: "array",
          items: { bsonType: "string" }
        },
        kurs_ids: {
          bsonType: "array",
          items: { bsonType: "string" }
        },
        credits: { bsonType: "int", minimum: 0 }
      }
    }
  }
});
```

Konsistenz und Transaktionen

- MongoDB unterstützt Transaktionen über mehrere Dokumente und Collections.
- Gute Modellierung versucht trotzdem, Multi-Document-Transaktionen zur Ausnahme zu machen.
- Je mehr Fachlogik mit verteilten Änderungen, desto teurer wird das System im Betrieb.

SQL vs MongoDB

- **SQL** ist stark bei stabilen, stark verknüpften Domänen mit vielen Joins.
- **MongoDB** ist stark bei dokumentnahen, hierarchischen und sich entwickelnden Datenstrukturen.
- SQL denkt stärker in Normalisierung und Beziehungen.
- MongoDB denkt stärker in Zugriffsmustern und fachlich zusammengehörigen Dokumenten.

Gute Einsatzszenarien für MongoDB

- Produkt- und Content-Kataloge mit variablen Attributen
- Event-, Telemetrie- und Log-Daten
- API-Backends mit JSON-lastigen Nutzdaten
- Anwendungen, in denen sich das Schema noch sichtbar weiterentwickelt

Wann relationale Systeme oft besser passen

- Viele komplexe Joins über stabile, stark verknüpfte Fachdomänen
- Harte Integritätsregeln über zahlreiche Entitäten hinweg
- Berichte, die stark von relationalen Auswertungen und klassischen BI-Mustern leben

Zum Schluss

- MongoDB ist besonders stark, wenn Daten fachlich natürlich als Dokumente auftreten.
- Gute MongoDB-Modellierung orientiert sich an Lese- und Schreibmustern, nicht zuerst an Normalformen.
- Flexibilität ist kein Freifahrtschein: Validierung, Indexe und Konsistenzentscheidungen bleiben Pflicht.

> Entscheidungsregel: Wenn ein fachlicher Vorgang meist auf ein Dokument oder wenige eng zusammengehörige Dokumente passt, ist MongoDB oft sehr stark.